

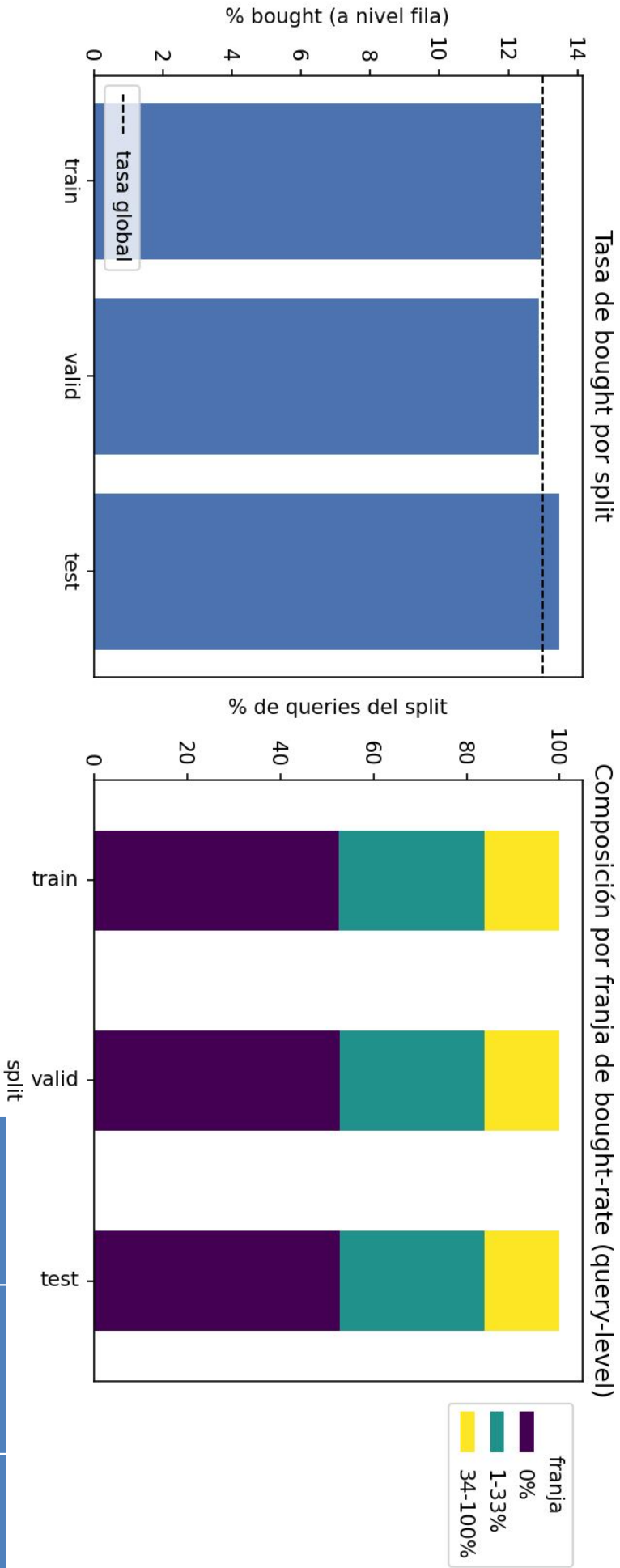
Ejercicio 2

Desarrollo del sistema

Split train / valid / test

- Cada búsqueda trae varios productos que comparten contexto (`filter_category`, `filter_storage_type`, `filter_price_min`, `filter_price_max`)
 - Si se particiona fila por fila, esas filas no son observaciones independientes entre sí — comparten esos filtros exactos
 - Repartirlas entre train y test podría inflar la métrica por esa correlación puntual entre filas, no por generalización real.
 - Partición por `query_id` completo: todas las filas de una búsqueda van al mismo split.
 - **Supuesto:** `query_id` se usa como clave de agrupación de condiciones de búsqueda compartidas, sin asumir que representa un evento puntual — `timestamp` no se usa ni para agrupar ni como feature.
-
- **train:** ajusta los pesos.
 - **valid:** compara configuraciones durante todo el estudio de experimentos
 - **test:** se toca una sola vez, al final, para el número reportado.

Split: 70/15/15 estratificado por tasa de bought



Trade-off:

- Train más grande: el dataset es chico en términos absolutos (2012 queries, 10.000 filas) y bought está desbalanceado (13%) — más train significa más ejemplos positivos reales para aprender el patrón de compra.
- Un valid/test más grande dan una representación más real de la tasa de bought

proporción	distancia promedio a la tasa global — valid	distancia promedio — test
70/15/15	0,32 puntos	0,28 puntos
80/10/10	0,40 puntos	0,35 puntos

Cómo evaluamos el modelo

PR-AUC (área bajo la curva Precisión-Recall)

- **Precisión:** $TP / (TP + FP)$ (verdaderos positivos sobre todo lo que el modelo marcó como positivo).
- **Recall:** $TP / (TP + FN)$ (verdaderos positivos sobre todos los que realmente eran positivos).
- PR-AUC resume qué tan bien el modelo separa lo comprado de lo no comprado
- El punto de comparación relevante no es 0 sino la prevalencia de la clase positiva

ROC-AUC (área bajo la curva ROC)

- **Tasa de verdaderos positivos:** $TP / (TP + FN)$
- **Tasa de falsos positivos:** $FP / (FP + TN)$
- Probabilidad de que, tomando un producto comprado y uno no comprado al azar, el modelo le dé mayor probabilidad al comprado

Overfitting: Gap de PR-AUC (train – valid)

Promedio sobre varias semillas: Cada configuración se corrió con 3 semillas (**media ± desvío estándar**)

Selección de **mejor época**: para cada semilla se elige la época con mayor PR-AUC de valid como el resultado de esa corrida

Arquitectura general del sistema

- Encoder-only para el texto (title + description): multi-head attention layer y una red feed-forward
- Encoder-Decoder completo: para secuencia-a-secuencia (ej. traducción) — no aplica.
- Decoder-only (como GPT): generación de texto — no aplica

Decisiones: Texto y datos tabulares se concatenan antes de la capa de salida. Razones:

- Permite desconectar el Transformer para análisis por separado, sin rediseñar la arquitectura.
- El texto tiene una estructura de orden que justifica usar multi head attention; lo tabular no tiene esa estructura secuencial

Setup de entrenamiento

parámetro	valor
Optimizador	Adam
Batch size	128
Semillas	0, 1, 2
Épocas	20

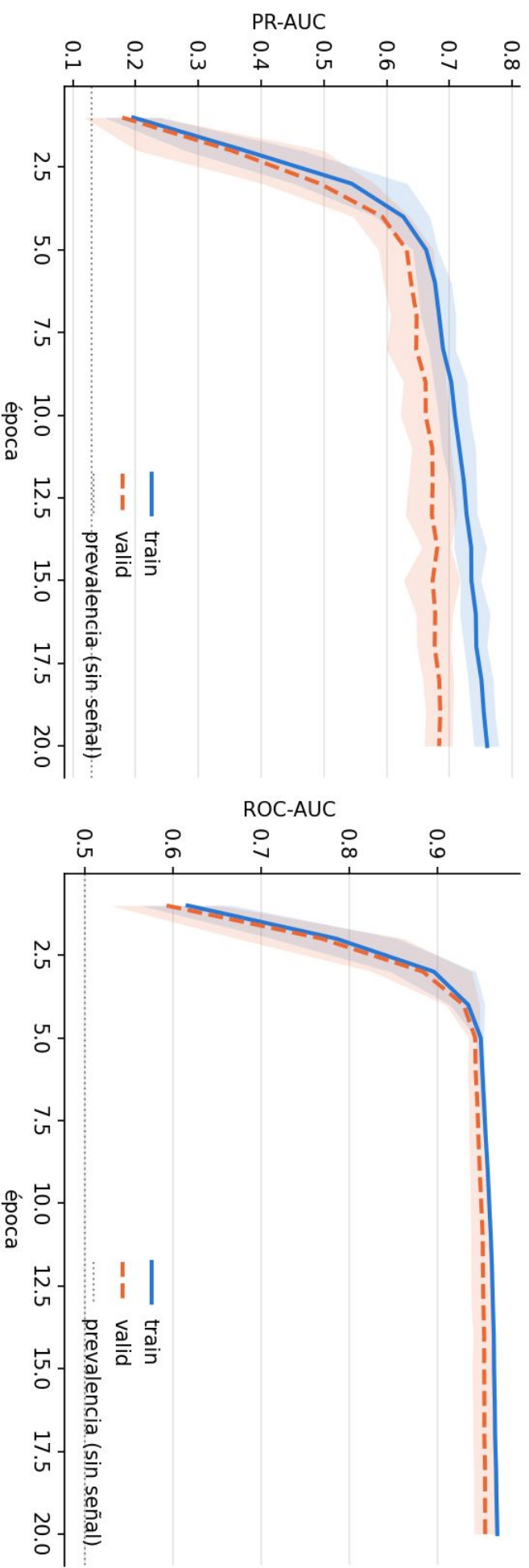
Experimento n°1

- Objetivo: aislar el comportamiento del Transformer

Arquitectura

- Embedding: d_model = 16 → empezamos chico
- n_heads = 1
- n_layers = 1 → un solo Encoder
- dim_feedforward = 64
- Positional encoding: senoidal → no agrega parámetros nuevos al modelo

Experimento 1 -- Transformer de texto puro (media ± desvío sobre 3 semillas)



Conclusiones

- Hay señal fuerte en el texto solo: PR-AUC (0,688) muy por encima de la prevalencia base de bought (0,130)
- Confirma lo visto en el ej 1: el tag de reputación embebido en title es predictivo
- Overfitting leve pero desaparejo entre semillas: el gap va de 0,024 a 0,114 (muestra la importancia de promediar corridas)
- En PR-AUC las curvas de train/valid se separan desde aprox la época 5
- En ROC-AUC casi no se separan (es menos sensible al desbalance de clases)

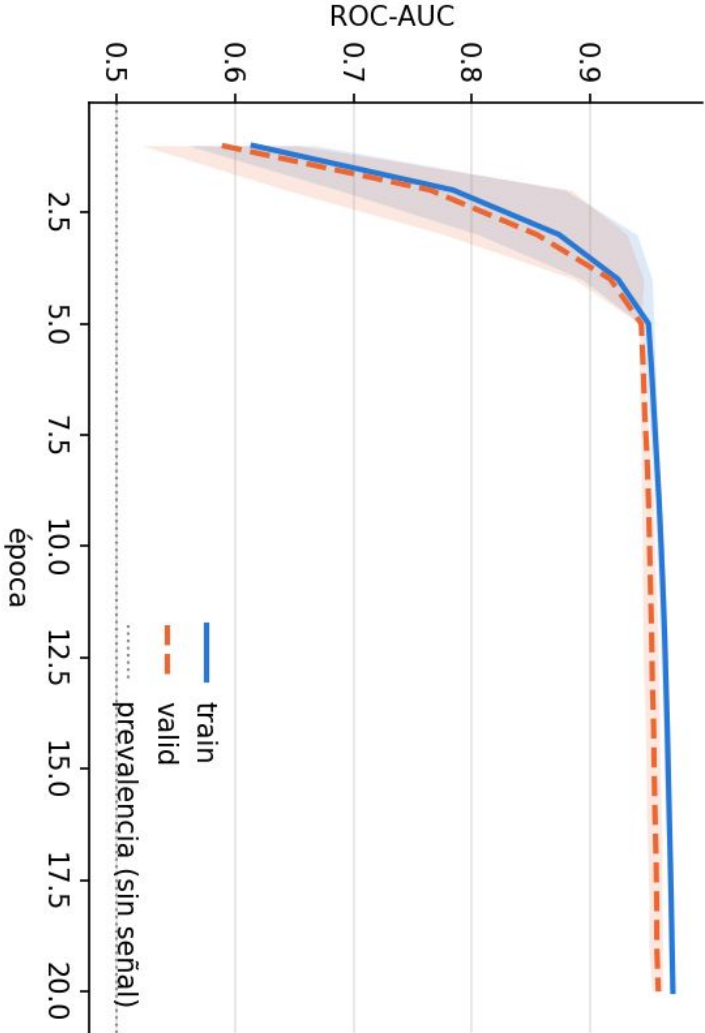
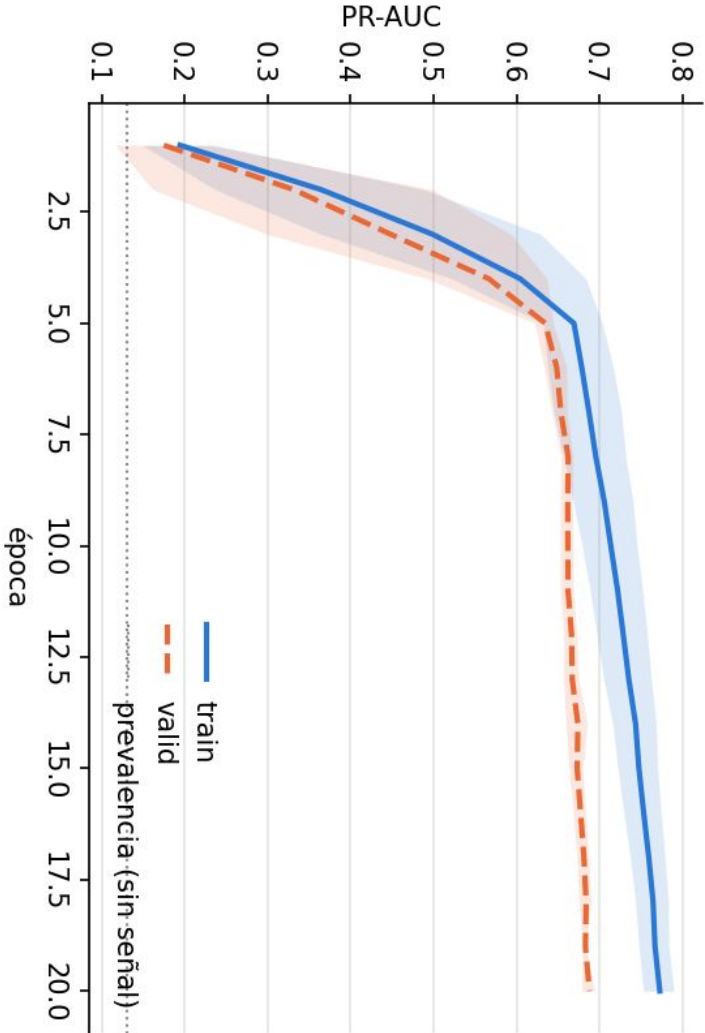
Experimento n°2 - aumento de heads

- Objetivo: Probar si separar distintos tipos de relación entre palabras ayuda a capturar mejor la señal, o si con un texto tan corto un solo head ya alcanza.

Arquitectura

- Embedding: d_model = 16
- **n_heads = 2**
- n_layers = 1
- dim_feedforward= 64
- Positional encoding: senoidal

Experimento 2 (media ± desvío sobre 3 semillas)



	Exp. 1 (n_heads=1)	Exp. 2 (n_heads=2)
valid PR-AUC	0,688 ± 0,024	0,688 ± 0,007
valid ROC-AUC	0,954 ± 0,012	0,957 ± 0,004
gap PR-AUC	0,058 ± 0,049	0,082 ± 0,015

- La performance pico no cambió (0,688 en ambos) → n_heads no agrega capacidad
- Cambió la estabilidad: el desvío estándar entre semillas bajó de 0,024 a 0,007
- En este rango de d_model, n_heads no es el cuello de botella de performance, es más un valor que afecta la estabilidad de entrenamiento que de capacidad.

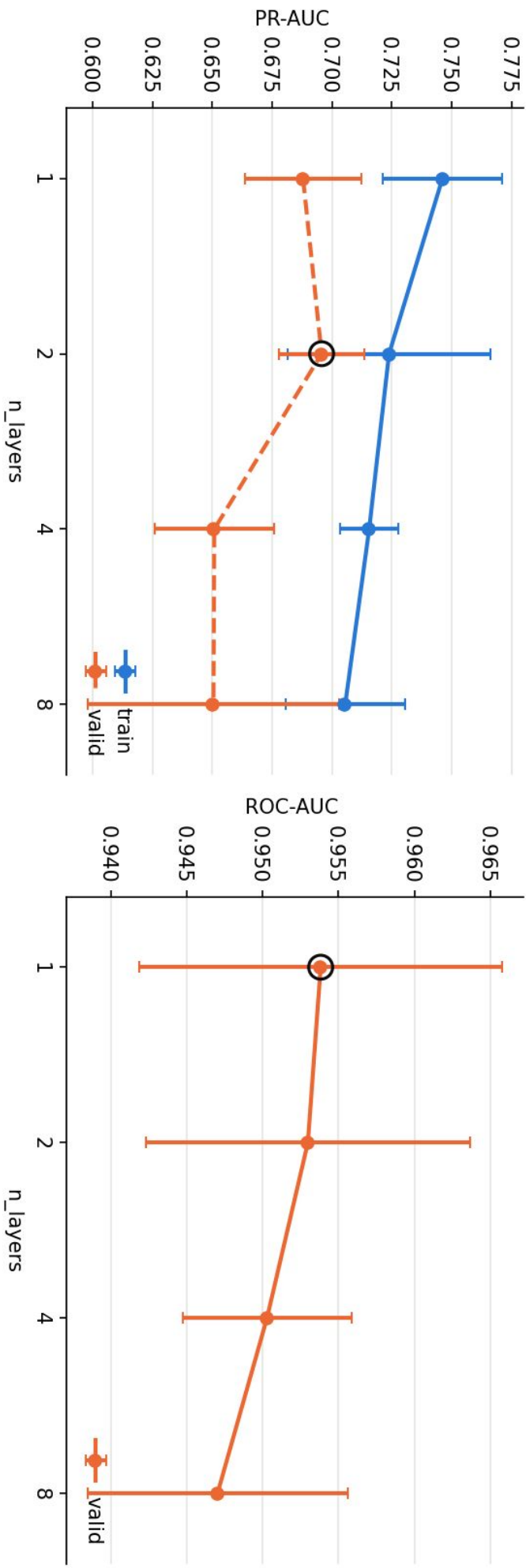
Experimento n°3 - layers

- Objetivo: aislar el comportamiento del Transformer

Arquitectura

- Embedding: $d_{\text{model}} = 16$
- $n_{\text{heads}} = 1$
- $n_{\text{layers}} = 1 / 2 / 4 / 8$
- $\text{dim}_{\text{feedforward}} = 64$
- Positional encoding: senoidal

Barrido de n_layers (círculo = mejor valor por PR-AUC de valid)



n_layers	valid PR-AUC	valid ROC-AUC	gap PR-AUC
1	0,688 ± 0,024	0,954 ± 0,012	0,058 ± 0,049
2	0,696 ± 0,018	0,953 ± 0,012	0,028 ± 0,042
4	0,651 ± 0,025	0,950 ± 0,006	0,065 ± 0,032
8	0,650 ± 0,053	0,947 ± 0,009	0,055 ± 0,035

- n_layers=2 gana en PR-AUC de valid y generaliza mejor
- En ROC-AUC, n_layers=1 queda apenas arriba
- Más profundidad no ayudó. Con un dataset chico y secuencias cortas, apilar más encoders agrega parámetros sin que haya más señal para aprovecharlos → modelo más difícil de optimizar de forma estable en pocas épocas.

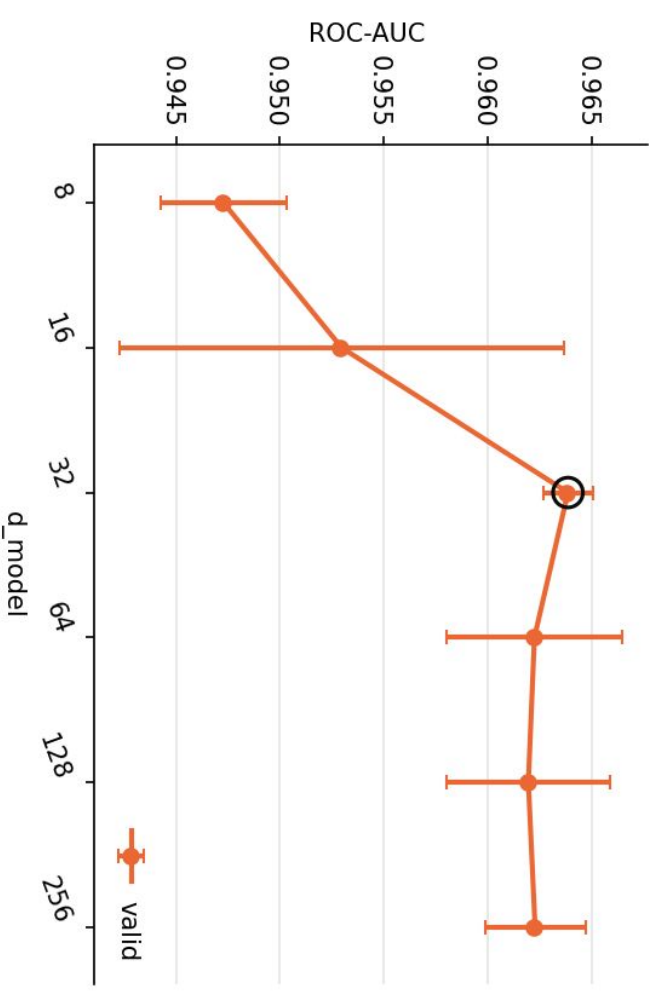
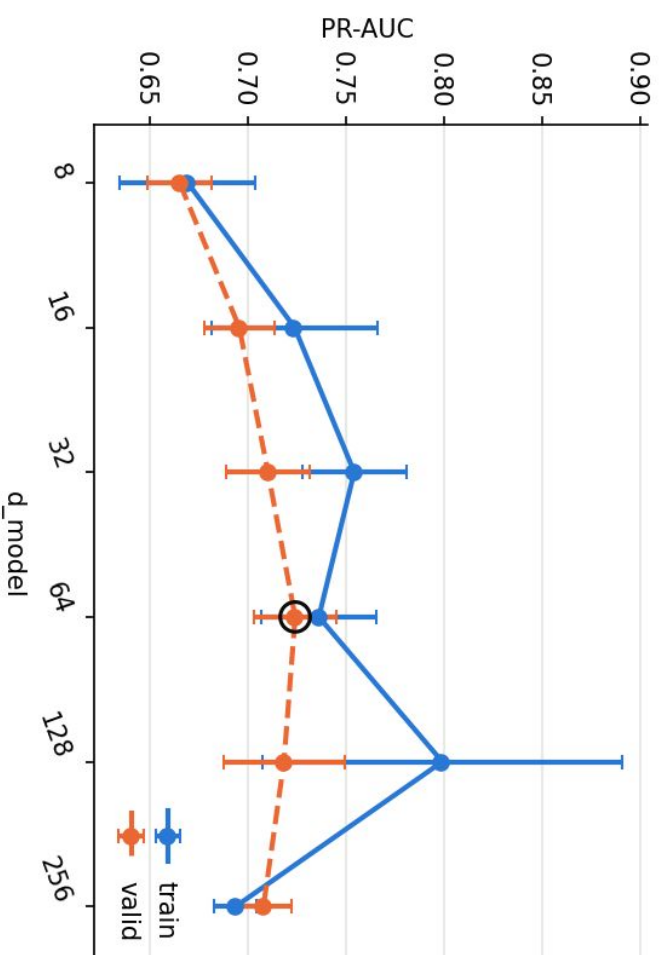
Experimento n°4 - d_model

- Objetivo: ver si el modelo venía corto de capacidad de representación

Arquitectura

- **Embedding: d_model = 8 / 16 / 32 / 64 / 128 / 256**
- n_heads = 1
- n_layers = 2
- dim_feedforward= 64
- Positional encoding: senoidal

Barrido de d_model (círculo = mejor valor por PR-AUC de valid)



d_model	valid PR-AUC	valid ROC-AUC	gap PR-AUC
8	0,665 ± 0,016	0,947 ± 0,003	0,004 ± 0,039
16	0,696 ± 0,018	0,953 ± 0,011	0,028 ± 0,042
32	0,710 ± 0,022	0,964 ± 0,001	0,044 ± 0,047
64	0,724 ± 0,021	0,962 ± 0,004	0,012 ± 0,014
128	0,718 ± 0,031	0,962 ± 0,004	0,081 ± 0,063
256	0,708 ± 0,014	0,962 ± 0,002	-0,014 ± 0,008

- PR-AUC sube en todo el rango probado, sin meseta ni caída
- d_model=64 es el mejor y el overfitting es chico (1x con respecto a dim_feedforward).
- En d_model=128 el gap de overfitting se dispara sin señal real adicional que aprovechar.
- En d_model=256 el gap da directamente negativo, el modelo ni siquiera logra ajustar bien train en 20 épocas

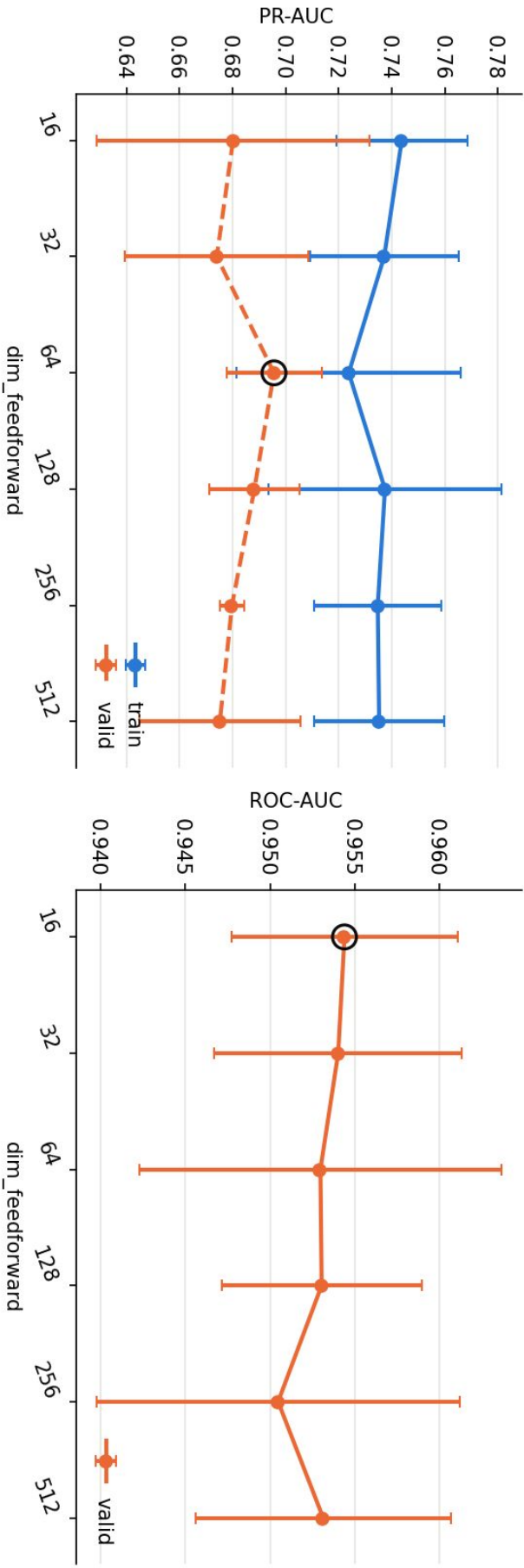
Experimento n°5 - **dim_feedforward**

- Objetivo: probar cambiar el ancho del MLP interno de cada encoder

Arquitectura

- Embedding: $d_{\text{model}} = 16$
- $n_{\text{heads}} = 1$
- $n_{\text{layers}} = 2$
- **$\text{dim}_{\text{feedforward}} = 16 / 32 / 64 / 128 / 256 / 512$**
- Positional encoding: senoidal

Barrido de dim_feedforward (círculo = mejor valor por PR-AUC de valid)



dim_feedforward	valid PR-AUC	valid ROC-AUC	gap PR-AUC
16	0,680 ± 0,051	0,954 ± 0,007	0,064 ± 0,044
32	0,674 ± 0,035	0,954 ± 0,007	0,063 ± 0,027
64	0,696 ± 0,018	0,953 ± 0,011	0,028 ± 0,042
128	0,688 ± 0,017	0,953 ± 0,006	0,049 ± 0,058
256	0,680 ± 0,005	0,951 ± 0,011	0,055 ± 0,023
512	0,675 ± 0,030	0,953 ± 0,008	0,060 ± 0,045

dim_feedforward=64 (proporción 4x sobre d_model=16) da mejor
Todos los valores por encima (128, 256, 512) quedan peor
Un feed-forward angosto es un cuello de botella y uno demasiado ancho agrega parámetros sin beneficio.

Experimento n°6 - d_model y dim_feedforward en conjunto

- Objetivo: analizar influencia de la proporción entre ambos parámetros. Ver si ese 4x importa de verdad o si alcanza con la capacidad absoluta de 64.

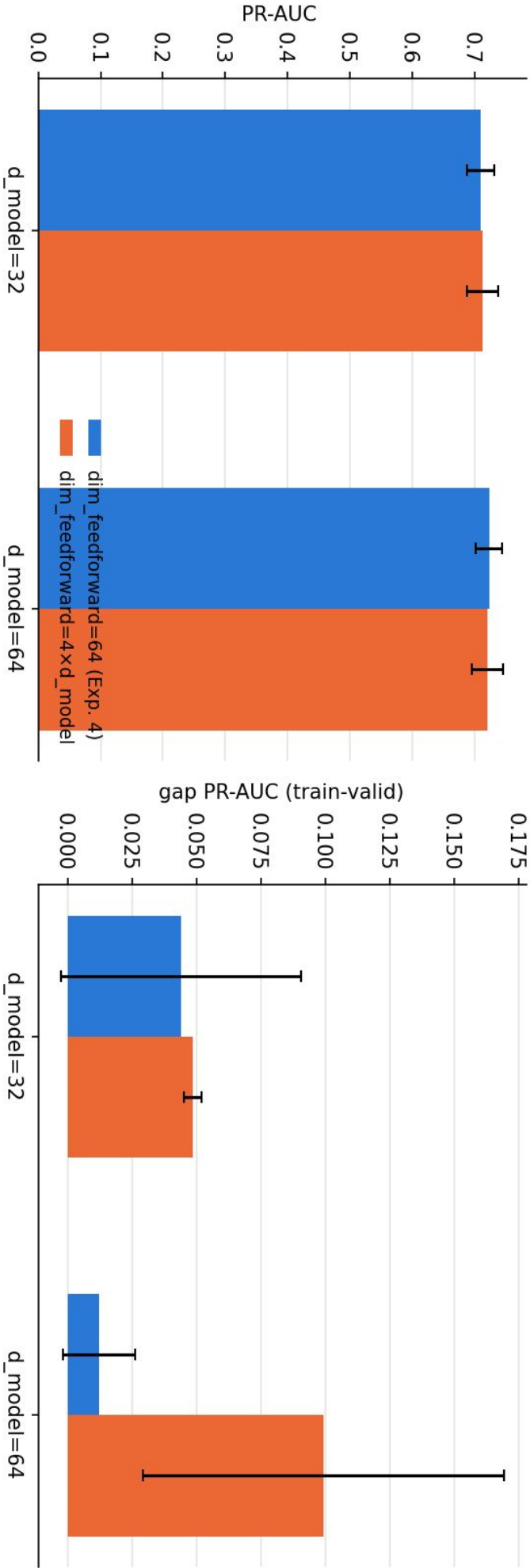
Arquitectura

- **Embedding: d_model = 8 / 16 / 32 / 64 / 128 / 256**
- n_heads = 1
- n_layers = 2
- **dim_feedforward= 16 / 32 / 64 / 128 / 256**
- Positional encoding: senoidal

d_model	dim_feedforward	proporción	valid PR-AUC	gap PR-AUC
32	64	2x	$0,710 \pm 0,022$	$0,044 \pm 0,047$
32	128	4x	$0,713 \pm 0,025$	$0,049 \pm 0,004$
64	64	1x	$0,724 \pm 0,021$	$0,012 \pm 0,014$
64	256	4x	$0,721 \pm 0,025$	$0,099 \pm 0,070$

- PR-AUC casi no cambia al escalar dim_feedforward en ninguno de los dos d_model, las diferencias están completamente dentro del desvío estándar de cada punto.
- Gráfico izquierdo: las barras son prácticamente idénticas en altura.
- Overfitting: en d_model=64, pasar de dim_feedforward=64 a 256 casi octuplica el gap de generalización ($0,012 \rightarrow 0,099$), con además mucha más varianza entre semillas (std 0,070).
- El feed-forward más ancho agrega capacidad que el modelo termina memorizando en vez de generalizar, sin que la performance de valid lo compense.
- Lo que pesa es la capacidad absoluta de d_model. Escalar también dim_feedforward no ayuda y en el punto más grande directamente perjudica la generalización.

Experimento 6 -- ¿escalar dim_feedforward junto con d_model ayuda?



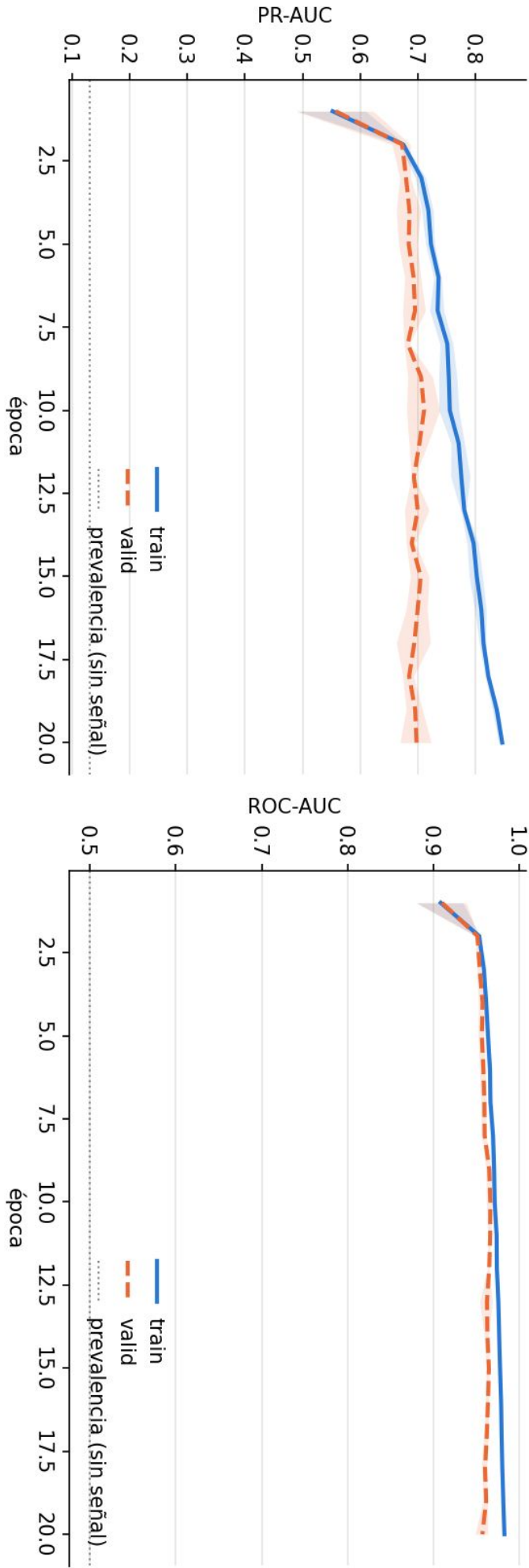
Experimento n°7 - sistema completo: texto + tabular

- Objetivo: analizar si concatenar el vector tabular al final mejora sobre el texto solo

Arquitectura

- Embedding: d_model = 64
- n_heads = 1
- n_layers = 2
- dim_feedforward= 64
- Positional encoding: senoidal

Experimento 7 (media ± desvío sobre 3 semillas)



	Solo texto	Texto + tabular
n_params	76.865	76.940
valid PR-AUC (media ± std)	0,724 ± 0,021	0,718 ± 0,023
valid ROC-AUC (media ± std)	0,962 ± 0,004	0,967 ± 0,002
gap PR-AUC (media ± std)	0,012 ± 0,014	0,045 ± 0,024

- Agregar lo tabular no mejoró PR-AUC.
- Con solo 75 features tabulares más y una sola capa lineal para combinarlas con el texto, el modelo no encontró en ellas señal adicional
- ROC-AUC sí mejoró, las tabulares ayudan a ordenar mejor los casos en general
- El overfitting empeoró bastante, y aparece mucho antes. El gap casi se cuadruplica (0,012 → 0,045) y la mejor época baja de ~15-20 a ~10-13
- **Hipótesis:** la capa de salida, un solo Linear, no ve relacion texto dato tabular

Experimento n°8 - capa oculta en la cabeza de salida

- Objetivo: Probar si el estancamiento del Experimento 7 se debía a la falta de una no-linealidad

Arquitectura

- Embedding: d_model = 64
- n_heads = 1
- n_layers = 2
- dim_feedforward= 64
- Positional encoding: senoidal

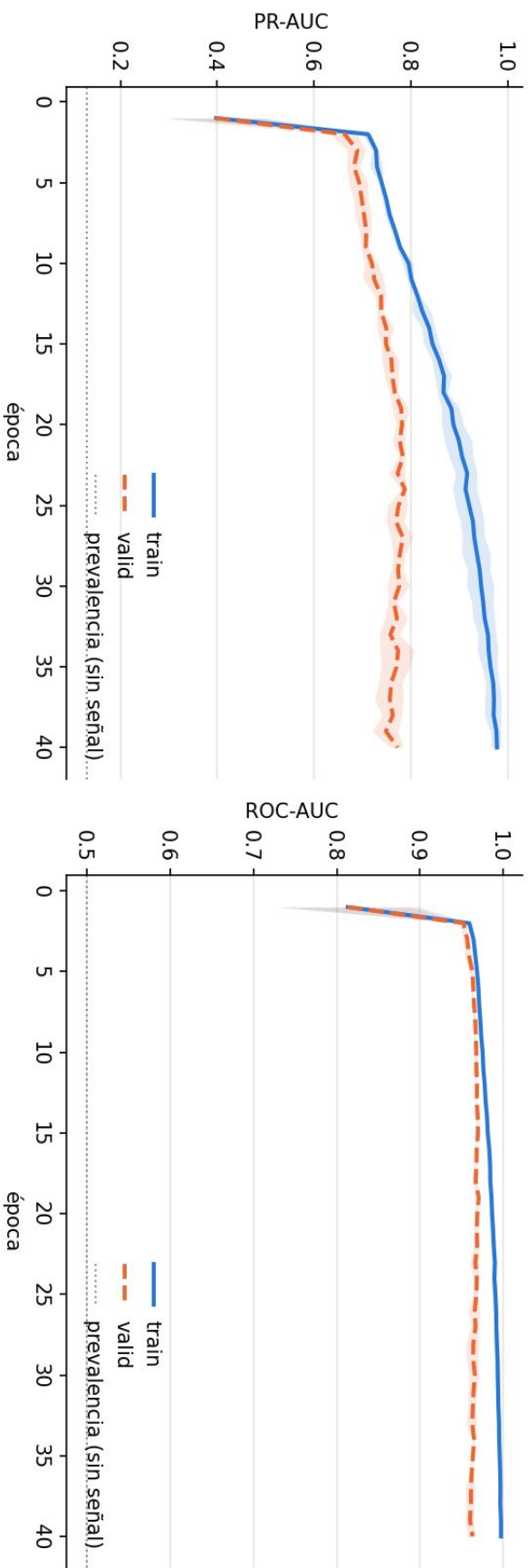
- Cabeza de salida: `Linear(139 → 64) → ReLU → Dropout(0,1) → Linear(64 → 1)`

Seguía subiendo sin señales de meseta, así que se repitió la misma configuración con **40 épocas**

Solo texto (Exp. 4/6)		Texto + tabular, sin capa oculta	Texto + tabular, con capa oculta
n_params	76.865	76.940	85.825
valid PR-AUC (media ± std)	0,724 ± 0,021	0,718 ± 0,023	0,798 ± 0,008
valid ROC-AUC (media ± std)	0,962 ± 0,004	0,967 ± 0,002	0,968 ± 0,003
gap PR-AUC (media ± std)	0,012 ± 0,014	0,045 ± 0,024	0,128 ± 0,021

- La no-linealidad dio la mejor marca de todos los experimentos
- Con 40 épocas se ve el techo que con 20 no se veía: valid PR-AUC sube fuerte hasta las épocas 20-25
- La ganancia corriendo 20 vs 40 épocas es chica (0,791 vs 0,798) y viene de picos puntuales de alguna semilla
- 20 épocas da un resultado representativo

Experimento 8 (media ± desvío sobre 3 semillas)



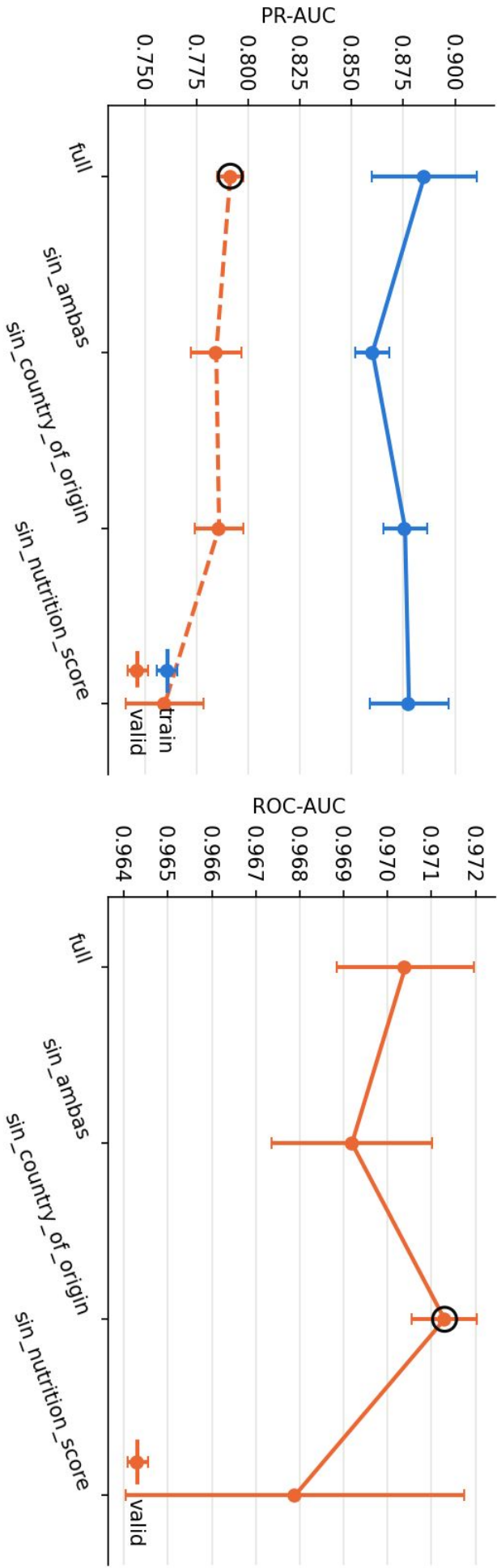
Experimento n°9 - **country_of_origin** y **nutrition_score**

- Objetivo: en el ejercicio 1 no encontramos señal clara para ninguna de las dos features.
Testear su importancia

Arquitectura

- Embedding: d_model = 64
- n_heads = 1
- n_layers = 2
- dim_feedforward= 64
- Positional encoding: senoidal
- Cabeza de salida: Linear(139 → 64) → ReLU → Dropout(0,1) → Linear(64 → 1)

Barrido de variant (círculo = mejor valor por PR-AUC de valid)



- Sacar country_of_origin apenas mueve el PR-AUC de valid (0,791 ± 0,006 → 0,786 ± 0,012, una diferencia menor que los propios desvíos) y baja un poco el gap de overfitting
- Sacar nutrition_score sí aporta señal real (0,791 ± 0,006 → 0,760 ± 0,019)
- **Decision:** se descarto country_of_origin achicando el modelo

Experimento n°10 - ancho de la capa oculta

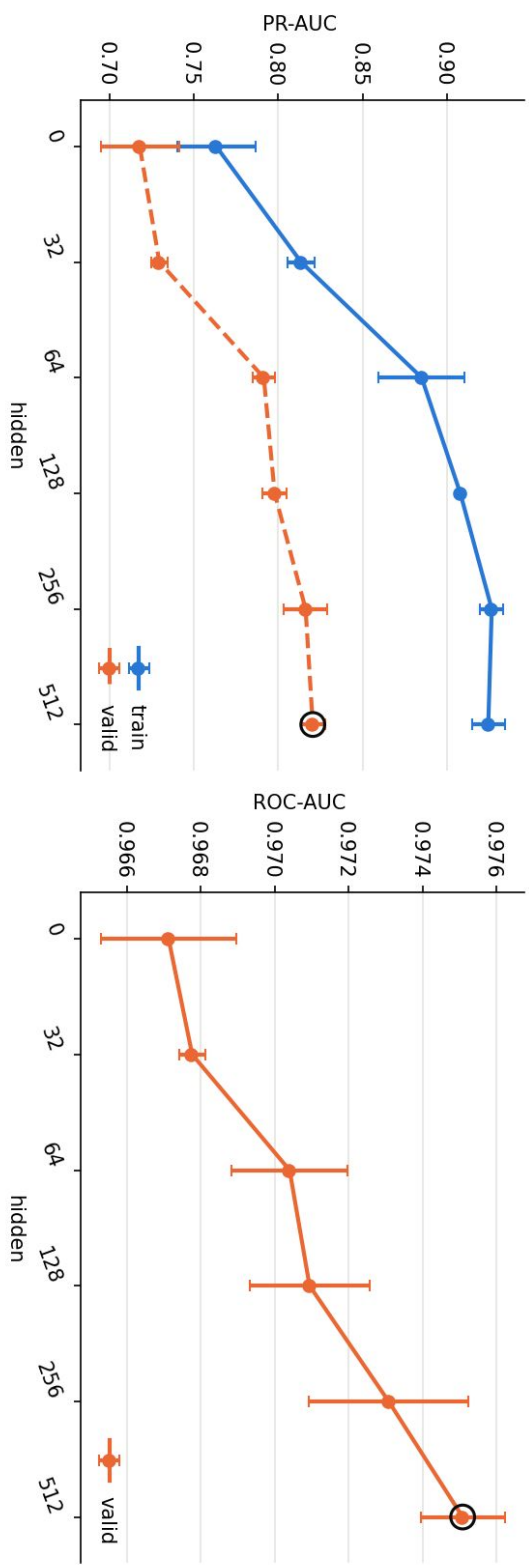
- Objetivo: Probar como variaciones del ancho de la capa oculta afectaba el rendimiento

Arquitectura

- Embedding: d_model = 64
- n_heads = 1
- n_layers = 2
- dim_feedforward= 64
- Positional encoding: senoidal

- Cabeza de salida: Linear(139 → hidden) → ReLU → Dropout(0,1) → Linear(hidden → 1)

Barrido de hidden (círculo = mejor valor por PR-AUC de valid)



hidden	n_params	valid PR-AUC	valid ROC-AUC	gap PR-AUC
0	76.940	0,718 ± 0,023	0,967 ± 0,002	0,045 ± 0,024
32	81.313	0,729 ± 0,005	0,968 ± 0,000	0,084 ± 0,010
64	85.825	0,791 ± 0,006	0,970 ± 0,002	0,094 ± 0,030
128	94.849	0,798 ± 0,007	0,971 ± 0,002	0,110 ± 0,007
256	112.897	0,816 ± 0,013	0,973 ± 0,002	0,111 ± 0,020
512	148.993	0,820 ± 0,007	0,975 ± 0,001	0,104 ± 0,003

- Tanto train como valid se aplanan entre 256 y 512, después de subir sostenido en todo el resto del barrido.
- hidden=512 es la mejor marca de todo el estudio, pero por un margen chico sobre hidden=256.
- Nos quedamos con hidden=256

Experimento n°11 - positional encoding

- Objetivo: Ver importancia del orden del texto

Arquitectura

- Embedding: d_model = 64
- n_heads = 1
- n_layers = 2
- dim_feedforward= 64
- Positional encoding: senoidal

variante	valid PR-AUC	train PR-AUC	gap PR-AUC
con positional encoding	0,724 ± 0,021	0,736	0,012 ± 0,014
sin positional encoding	0,720 ± 0,001	0,893	0,174 ± 0,022

- PR-AUC de valid casi no cambia (0,724 → 0,720) — sacar el positional encoding no le hace perder al modelo casi nada de capacidad de alcanzar un buen resultado en valid.
- Tag de reputación en title (un patrón que parecía "posicional"), ese patrón puede ser más una cuestión de qué palabras aparecen juntas que de en qué posición exacta aparecen.
- Overfitting: el gap se dispara de 0,012 a 0,174
- **Decisión:** se mantiene positional encoding en la arquitectura final porque sin él el modelo sobreajusta muchísimo más rápido y más fuerte, lo cual sería un problema real en un dataset todavía más chico o entrenando más épocas.

Modelo final elegido

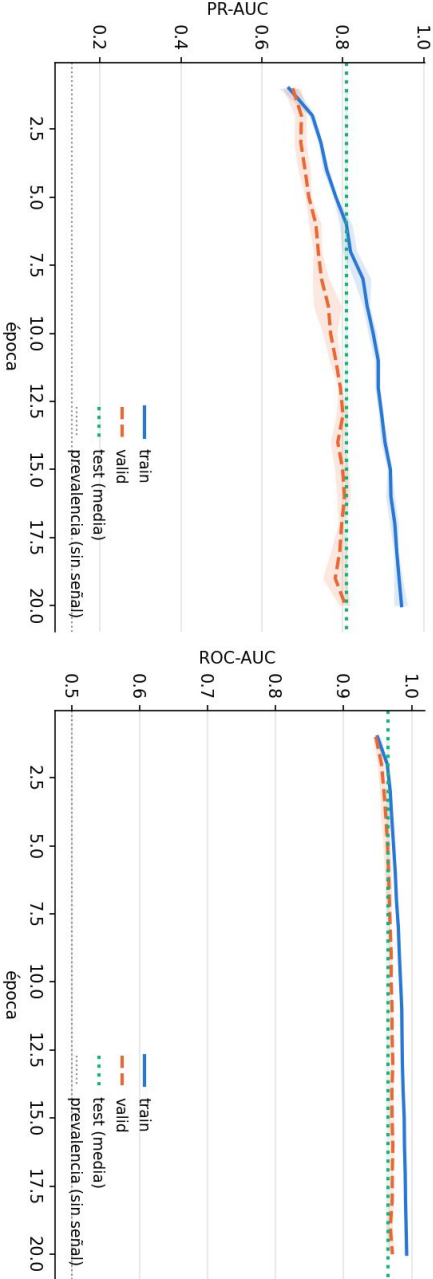
Arquitectura

- Embedding: $d_{\text{model}} = 64$
- $n_{\text{heads}} = 1$
- $n_{\text{layers}} = 2$
- $\text{dim}_{\text{feedforward}} = 64$
- Positional encoding: senoidal
- Cabeza de salida: $\text{Linear}(129 \rightarrow 256) \rightarrow \text{ReLU} \rightarrow \text{Dropout}(0.1) \rightarrow \text{Linear}(256 \rightarrow 1)$

valid PR-AUC	valid ROC-AUC	gap PR-AUC
0,816 $\pm$ 0,013	0,973 $\pm$ 0,002	0,111 $\pm$ 0,020

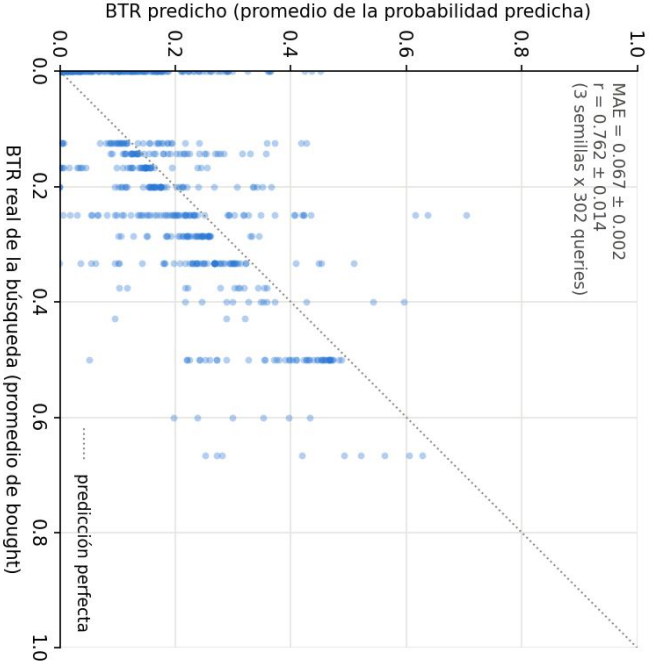
Evaluación en test

Configuración final -- train/valid por época + resultado de test (3 semillas)



seed	mejor época (por valid)	test PR-AUC	test ROC-AUC
0	16	0,806	0,962
1	13	0,813	0,966
2	15	0,809	0,967
media ± std	—	0,809 ± 0,003	0,965 ± 0,003

BTR por búsqueda en test: real vs. predicho



- El modelo agarra bien la tendencia general ($r=0,76$) y el error típico por búsqueda es chico (0,067).
- Tiende a sobreestimar el BTR, se ve la columna de puntos en $btr_real=0$ con predicciones dispersas (el modelo se entrena a nivel fila)